

Ordonnabilité des espaces métriques

Le sujet comporte 12 pages, numérotées de 1 à 12.

Début de l'épreuve.

Dans ce sujet, on s'intéresse au problème d'ordonner les éléments d'un espace métrique de sorte que deux éléments successifs soient à distance bornée.

Ce sujet est constitué de quatre parties. La première partie est formée de préliminaires utiles dans le reste du sujet, avec des questions de programmation en C et OCaml. La deuxième partie propose une implémentation en C d'une structure de données utilisée notamment dans la troisième partie. Cette troisième partie considère les graphes, munis de la distance de plus court chemin, comme espace métrique, avec implémentations en C. La quatrième partie, enfin, indépendante des deux précédentes, considère l'ordonnabilité des langages réguliers pour une certaine distance d'édition ; cette partie contient de la programmation en OCaml.

Dans les questions de programmation en C ou OCaml, on n'utilisera pas de fonctions qui ne sont pas incluses dans la bibliothèque standard du langage. Pour les codes en C, on pourra supposer que les en-têtes `<stdbool.h>`, `<stdio.h>`, `<stdlib.h>` et `<assert.h>` ont été inclus.

Espaces métriques et d -ordres

Dans ce sujet, on s'intéresse à des collections de données discrètes que l'on cherche à explorer ou engendrer de proche en proche. On modélise ce cadre général à l'aide des notions d'*espace métrique*, de *d -suite* et de *d -ordre* :

Définition 1 (espace métrique). *Un espace métrique $\mathcal{M} = (X, \delta)$ est constitué d'un ensemble dénombrable X dont les éléments sont appelés points et d'une distance sur X , c'est-à-dire une application $\delta : X \times X \rightarrow \mathbb{N}$ satisfaisant :*

Symétrie. Pour tout $(x, y) \in X^2$, $\delta(x, y) = \delta(y, x)$.

Séparation. Pour tout $(x, y) \in X^2$, $\delta(x, y) = 0$ si et seulement si $x = y$.

Inégalité triangulaire. Pour tout $(x, y, z) \in X^3$, $\delta(x, y) \leq \delta(x, z) + \delta(z, y)$.

Pour un espace métrique $\mathcal{M} = (X, \delta)$ et $X' \subseteq X$, on note $\mathcal{M}[X']$ le couple $(X', \delta|_{X'})$, où $\delta|_{X'}$ dénote la restriction de δ au domaine $X' \times X'$. On observe que $\mathcal{M}[X']$ est encore un espace métrique, que l'on appellera sous-espace de $\mathcal{M}$.

Définition 2 (d -suite). *Pour $d \in \mathbb{N}^*$, une d -suite $\mathbf{s}$ dans un espace métrique $\mathcal{M} = (X, \delta)$ est une suite (finie ou infinie dénombrable) $\mathbf{s} = x_1, x_2, \dots$ de points de $\mathcal{M}$ telle que :*

- $\mathbf{s}$ ne contient pas de doublons, c'est-à-dire que pour tous x_i, x_j de la suite, si $i \neq j$ alors $x_i \neq x_j$;
- deux points consécutifs de la suite sont à distance au plus d ; formellement, pour tous x_i, x_{i+1} de la suite on a $\delta(x_i, x_{i+1}) \leq d$.

On dit que $\mathbf{s}$ commence en x_1 et, dans le cas où $\mathbf{s}$ est finie et a n éléments, que $\mathbf{s}$ termine en x_n .

Définition 3 (d -ordre, d -ordonnable). *Pour $d \in \mathbb{N}^*$, un d -ordre de $\mathcal{M}$ est une d -suite dans $\mathcal{M}$ contenant tous les points de $\mathcal{M}$. L'espace $\mathcal{M}$ est dit d -ordonnable lorsqu'il existe un d -ordre de $\mathcal{M}$. On dit que $\mathcal{M}$ est ordonnable lorsque $\mathcal{M}$ est d -ordonnable pour un certain d .*

Distances d'édition sur les mots

On fixe dans tout le sujet l'*alphabet* $\Sigma := \{a, b\}$ ayant pour seules lettres a et b . Un *mot* w est une suite finie de lettres $w = \alpha_1 \cdots \alpha_n$. La longueur de w , notée $|w|$, est n . On note ε le mot vide, de longueur nulle. On note Σ^* l'ensemble des mots sur Σ . Un *langage* est un sous-ensemble de Σ^* .

Un espace métrique d'intérêt, sur l'ensemble des mots d'un langage, est donné par deux distances d'édition sur les mots : la distance *push-pop* et la distance *push-pop-droite*, que l'on définit ci-après.

Définition 4 (distance push-pop). *La distance push-pop, dénotée δ_{pp} , est définie de la manière suivante : pour $w, w' \in \Sigma^*$, $\delta_{pp}(w, w')$ est le nombre minimal d'opérations nécessaires pour passer de w à w' , où les opérations autorisées sont :*

- pour un mot w , insérer la lettre $\alpha \in \Sigma$ à la fin, ce qui donne le mot $w\alpha$;
- pour un mot w , insérer la lettre $\alpha \in \Sigma$ au début, ce qui donne le mot αw ;
- pour un mot de la forme $w\alpha$ avec $\alpha \in \Sigma$, supprimer la dernière lettre, ce qui donne le mot w ;
- pour un mot de la forme αw avec $\alpha \in \Sigma$, supprimer la première lettre, ce qui donne le mot w .

Exemple 1. *Les mots à distance push-pop 1 du mot aab sont aa (on a supprimé la dernière lettre), $aaba$ (on a ajouté un a à la fin), $aabb$ (on a ajouté un b à la fin), ab (on a supprimé la première lettre), $aaab$ (on a ajouté un a au début) et $baab$ (on a ajouté un b au début).*

Définition 5 (distance push-pop-droite). *La distance push-pop-droite, dénotée δ_{ppr} , est définie de la même manière que δ_{pp} mais seules les insertions et suppressions à la fin du mot sont autorisées.*

Exemple 2. *Les mots qui sont à distance push-pop-droite 1 du mot aab sont aa (on a supprimé la dernière lettre), $aaba$ (on a ajouté un a à la fin) et $aabb$ (on a ajouté un b à la fin).*

Algorithmes d'énumération push-pop et push-pop-droite

Lorsqu'on travaillera sur les mots, on considèrera parfois des programmes produisant une suite (potentiellement infinie) de mots de Σ^* , en utilisant les instructions spéciales suivantes : **popL()**, **popR()** ; **pushL(α)** et **pushR(α)** pour $\alpha \in \Sigma$; et **output()**.

Le programme dispose, comme état interne, d'une liste L d'éléments de Σ , qui est interprétée comme un mot de Σ^* . La liste L est initialement vide et représente donc le mot vide. Voici ce qu'il se passe lorsque le programme utilise les instructions spéciales :

- **popL()** a pour effet de supprimer la première lettre de la liste (et ne peut être appelée que si la liste est non vide) ;
- **popR()** a pour effet de supprimer la dernière lettre de la liste (et ne peut être appelée que si la liste est non vide) ;
- **pushL(α)** a pour effet d'ajouter la lettre $\alpha \in \Sigma$ en début de liste ;
- **pushR(α)** a pour effet d'ajouter la lettre $\alpha \in \Sigma$ en fin de liste ;
- **output()** *produit* le mot étant actuellement représenté par la liste (par exemple, il l'affiche en sortie du programme).

On souligne que la liste L n'est accessible par le programme que via ces instructions spéciales. De plus, on fait l'hypothèse que chacune des instructions **popL**, **popR**, **pushL** et **pushR** a une complexité en $O(1)$.

Un tel programme *produit* alors la suite de mots $w_1, w_2, \dots$, où w_1 est le premier mot produit par le programme lors de son exécution, w_2 le second et ainsi de suite.

On appellera un tel programme un *programme push-pop*. De manière similaire, un programme *push-pop-droite* est un programme push-pop qui n'utilise pas les instructions **popL()** ni **pushL(α)** pour $\alpha \in \Sigma$.

Exemple 3. On suppose que les fonctions `C` `pushL`, `pushR`, `popL`, `popR`, `output` ont été prédéfinies et manipulent toutes une même variable représentant la liste. Le programme `C` suivant est un programme push-pop-droite qui produit la suite de mots $w_1, w_2, \dots$ où w_i est $(aa)^{i-1}b$:

```
int main(void)
{
  pushR('b'); output();
  while (true) {
    popR(); pushR('a'); pushR('a'); pushR('b'); output();
  }
}
```

On remarque que ce programme ne termine jamais.

En OCaml, on suppose qu'on dispose d'un type `lettre` défini par :

```
type lettre = A | B
```

On représente alors les mots de Σ^* par des listes OCaml `lettre list`, dont la tête correspond à la première lettre du mot. Par exemple, le mot *aab* est représenté par la liste `[A;A;B]`. On suppose avoir accès en OCaml à des fonctions de type `pushR : lettre -> unit`, `pushL : lettre -> unit`, `popR : unit -> unit`, `popL : unit -> unit`, ainsi que `output : unit -> unit`, qui représentent les instructions spéciales.

Exemple 4. Le programme OCaml push-pop suivant produit la même suite que le programme `C` push-pop-droite de l'exemple 3 :

```
let main () =
  pushR B; output ();
  while true do
    pushL A; pushL A; output ();
  done
```

Partie I : Préliminaires

Question I.1. Soit $\mathcal{M} = (X, \delta)$ un espace métrique tel que X est un ensemble fini. Montrer que $\mathcal{M}$ est ordonnable.

Question I.2. Écrire une fonction OCaml

`delta_ppr : lettre list -> lettre list -> int`

prenant en entrée deux listes `l1`, `l2` représentant des mots w_1, w_2 et renvoyant $\delta_{\text{ppr}}(w_1, w_2)$, la distance push-pop-droite entre w_1 et w_2 . On attend de cette fonction que sa complexité soit linéaire en $|w_1| + |w_2|$.

Question I.3. On considère la suite $\mathbf{s} := w_1, w_2, \dots$, où w_i est a^{i^2} . Écrire un programme push-pop-droite en C qui produit la suite $\mathbf{s}$.

Question I.4. Montrer que $\mathcal{M}_{\text{pp}} := (\Sigma^*, \delta_{\text{pp}})$ et $\mathcal{M}_{\text{ppr}} := (\Sigma^*, \delta_{\text{ppr}})$ sont des espaces métriques.

On s'intéresse maintenant aux sous-espaces de $\mathcal{M}_{\text{pp}}$ et $\mathcal{M}_{\text{ppr}}$, c'est-à-dire (comme indiqué dans la Définition 1) aux espaces métriques de la forme $\mathcal{M}_{\text{pp}}[L] = (L, \delta_{\text{pp}|L})$ ou $\mathcal{M}_{\text{ppr}}[L] = (L, \delta_{\text{ppr}|L})$, pour L un langage. Par exemple, la suite produite par les programmes push-pop des exemples 3 et 4 est un 4-ordre pour $\mathcal{M}_{\text{ppr}}[L]$ et un 2-ordre pour $\mathcal{M}_{\text{pp}}[L]$, où $L := (aa)^*b$.

Question I.5. Écrire un programme push-pop en C qui produit un d -ordre pour $\mathcal{M}_{\text{pp}}[L]$, où $L := a^*b^* \mid b^*a^*$ et un certain $d \in \mathbb{N}$. Expliquer comment fonctionne ce programme.

Question I.6. Écrire un programme push-pop en C qui produit un 1-ordre pour $\mathcal{M}_{\text{pp}}[L]$, où $L := a^*b^*$. [*Indice* : on peut visualiser le langage L comme la grille $\mathbb{N} \times \mathbb{N}$, où à la position (i, j) correspond le mot $a^i b^j$.] Ne pas hésiter à faire un dessin pour se faire comprendre.

Question I.7. Soit $L := b^* \mid ab^*$.

- Écrire un programme push-pop en C qui produit un 1-ordre pour $\mathcal{M}_{\text{pp}}[L]$.
- Prouver que $\mathcal{M}_{\text{ppr}}[L]$ n'est pas ordonnable.

Question I.8. Un *chemin hamiltonien* dans un graphe non-orienté est un chemin (aussi appelé *chaîne*) qui visite tous les sommets du graphe exactement une fois. Un *graphe grille* est un graphe de la forme $G = (V, E_V)$ avec V une partie finie de $\mathbb{N} \times \mathbb{N}$ et $E_V = \{(i, j), (i', j')\} \mid (i, j), (i', j') \in V \text{ et } |i - i'| + |j - j'| = 1\}$, autrement dit, deux sommets sont connectés par une arête s'ils sont à distance 1 dans la grille $\mathbb{N} \times \mathbb{N}$.

On définit le problème de décision **HamGrille** de la manière suivante : en entrée on a un graphe grille G et la sortie est OUI si G possède un chemin hamiltonien et NON sinon. On admettra que ce problème est NP-complet.

Pour t un entier naturel non nul, on définit le problème de décision $\mathcal{P}_t$, de la façon suivante : $\mathcal{P}_t$ prend en entrée un ensemble fini de mots L et la sortie est OUI si $\mathcal{M}_{\text{pp}}[L]$ est t -ordonnable, NON sinon. On fixe maintenant un entier naturel $t \geq 1$ arbitraire. Montrer que pour cet entier t , le problème $\mathcal{P}_t$ est NP-complet.

Partie II : Implémentation en C des programmes push-pop

Dans cette partie, on souhaite réaliser une implémentation en langage C des opérations des programmes push-pop sur une structure de données `liste` codant une liste doublement chaînée. Le type de la structure de données `liste` est défini comme suit :

```
typedef struct chainon_s chainon;
```

```
struct chainon_s
{
    int val;
    chainon *prec;
    chainon *suiv;
};
```

```
struct liste_s
{
    chainon *premier;
    chainon *dernier;
};
```

```
typedef struct liste_s liste;
```

En particulier, les lettres sont donc représentées par des entiers (on rappelle qu'en C, une variable de type `char` peut être implicitement interprétée comme variable de type `int` sans perte d'informations); cela permettra de réutiliser cette structure `liste` dans d'autres contextes.

Dans l'implémentation de cette structure de données, on veillera à ce qu'un pointeur dont la valeur n'est pas `NULL` pointe toujours vers un espace mémoire valide et pas, par exemple, vers une donnée libérée.

Question II.1. Définir et initialiser une variable globale `lg` représentant la liste chaînée globale manipulée par les programmes push-pop; on souhaite que cette liste soit initialement vide.

Question II.2. Programmer une fonction `est_vide` de prototype `bool est_vide(void)` renvoyant `true` si `lg` représente une liste vide, `false` sinon.

Question II.3. Programmer des fonctions `pushL` et `pushR` de prototypes `void pushL(int)` et `void pushR(int)` implémentant les opérations `pushL` et `pushR` sur la liste représentée par `lg`. On utilisera une assertion pour vérifier que l'allocation dynamique de mémoire est bien réalisée. Quelle est la complexité en temps de ces deux fonctions ?

Question II.4. Programmer des fonctions `popL` et `popR` de prototypes `int popL(void)` et `int popR(void)` implémentant les opérations `popL` et `popR` sur la liste représentée par `lg` (chacune renvoyant également la valeur extraite de la liste). On utilisera une assertion pour s'assurer que la liste n'est pas vide et on veillera à libérer la mémoire devenue inutile. Quelle est la complexité en temps de ces deux fonctions ?

Question II.5. Programmer des fonctions `output` et `vide_liste` de prototypes respectifs `void output(void)` et `void vide_liste(void)`; `output` affiche le mot codé par la liste de caractères sur la sortie standard (suivi d'un retour à la ligne), tandis que `vide_liste` vide la liste (retire tous ses éléments) et libère la mémoire dynamique associée. Quelle est la complexité en temps de ces deux fonctions ?

**ECOLE POLYTECHNIQUE
ECOLES NORMALES SUPERIEURES**

CONCOURS D'ADMISSION 2023

**MARDI 18 AVRIL 2023
14h00 - 18h00**

**FILIERE MP-MPI - Epreuve n° 4
INFORMATIQUE A (XULSR)**

Durée : 4 heures

***L'utilisation des calculatrices n'est pas autorisée pour
cette épreuve***

*Cette composition ne concerne qu'une partie des candidats de la
filière MP, les autres candidats effectuant simultanément la
composition de Physique et Sciences de l'Ingénieur.
Pour la filière MP, il y a donc deux enveloppes de Sujets pour
cette séance.*

Compression entropique

Le sujet comporte 11 pages, numérotées de 1 à 11.

Vue d'ensemble du sujet.

Ce sujet s'intéresse à la compression et à la décompression de mots sur un alphabet fini $\mathcal{S}$ de cardinal $|\mathcal{S}| \geq 2$. Plus précisément, on se donne une fonction q de $\mathcal{S}$ dans $\mathbb{N}$ et on s'intéresse aux mots de $\mathcal{S}^*$ tels que chaque lettre $\sigma \in \mathcal{S}$ a exactement $q(\sigma)$ occurrences dans ces mots. On note $\mathcal{S}^q$ l'ensemble de ces mots. Remarque : les mots de $\mathcal{S}^q$ ont pour longueur $N = \sum_{\sigma \in \mathcal{S}} q(\sigma)$.

Une lettre de $\mathcal{S}$ peut être représentée avec $\lceil \log_2 |\mathcal{S}| \rceil$ bits, où $\lceil x \rceil$ désigne la partie entière supérieure de x , c'est-à-dire l'entier tel que $\lceil x \rceil - 1 < x \leq \lceil x \rceil$. Si l'on concatène les bits représentant chacune des lettres d'un mot de $\mathcal{S}^q$, on peut donc représenter ce mot de façon non ambiguë à l'aide d'une séquence de bits de taille

$$\sum_{\sigma \in \mathcal{S}} q(\sigma) \cdot \lceil \log_2 |\mathcal{S}| \rceil = \lceil \log_2 |\mathcal{S}| \rceil \cdot N.$$

Il s'agit d'une représentation non compressée.

La théorie de l'information affirme qu'il faut au moins $\sum_{\sigma \in \mathcal{S}} q(\sigma) \cdot \log_2(N/q(\sigma))$ bits pour représenter tous les mots de $\mathcal{S}^q$ de façon non ambiguë. Ce sujet explore quelques façons de compresser les mots pour s'approcher de cette borne théorique.

La partie I s'intéresse à la fonction q , c'est-à-dire au comptage des lettres d'un mot. La partie II étudie les arbres binaires dont les feuilles sont étiquetées par des lettres de $\mathcal{S}$. La partie III utilise ces arbres pour (dé)compresser des mots de $\mathcal{S}^*$. La partie IV s'intéresse aux arbres qui donnent les meilleurs taux de compression pour les mots de $\mathcal{S}^q$. Certains de ces arbres ont une représentation compacte ; c'est l'objet de la partie V. D'autres arbres sont encore plus compacts, mais au prix d'un moindre taux de compression, comme le montre la partie VI. Finalement, la partie VII attaque le problème d'une façon complètement différente afin de s'approcher un peu plus du taux de compression théorique optimal.

Les différentes parties sont indépendantes : il n'est pas nécessaire d'avoir répondu aux questions d'une partie pour répondre aux questions d'une autre partie ; cependant les notions abordées dans une partie sont souvent utiles aux parties suivantes.

Rappels d'OCaml

On rappelle ici quelques opérations de base sur les tableaux :

- `Array.length t` renvoie la longueur du tableau `t`.
- `Array.make n v` crée un tableau de `n` cases qui sont toutes initialisées avec `v`.
- `Array.of_list l` renvoie un tableau initialisé avec les valeurs de la liste `l`, tandis que `Array.to_list t` réalise l'opération inverse.
- La case numéro `i` du tableau `t` peut être accédée avec `t.(i)`. Les cases sont numérotées à partir de zéro.

Partie I. Comptage d'occurrences

Un texte non compressé est un mot de $\mathcal{S}^*$. Dans la suite, à chaque fois qu'il s'agira de définir une fonction en OCaml, les lettres de $\mathcal{S}$ seront représentées par des entiers et un mot de $\mathcal{S}^*$ sera représenté par un tableau d'entiers (`int array`) ou une liste d'entiers (`int list`) en fonction des questions. La première étape consiste à calculer le nombre d'occurrences $q(\sigma)$ de chaque lettre σ dans un mot.

Question I.1. Supposons $\mathcal{S} = [0, 255]$. Définissez une fonction OCaml `occurrences : int list -> int array` qui reçoit en argument un mot représenté par une liste d'entiers et renvoie le nombre d'occurrences $q(\sigma)$ de chaque lettre $\sigma \in \mathcal{S}$ sous forme d'un tableau $\llbracket q(0); q(1); \dots; q(255) \rrbracket$.

Il peut être intéressant de ne considérer que le sous-ensemble des lettres qui ont au moins une occurrence. Dans la question suivante, on ne représentera pas q par $\llbracket q(0); q(1); \dots; q(|\mathcal{S}| - 1) \rrbracket$ mais par un tableau de paires $\llbracket (\sigma_1, q(\sigma_1)); (\sigma_2, q(\sigma_2)); \dots \rrbracket$ avec $\{\sigma_1, \sigma_2, \dots\} = \{\sigma \in \mathcal{S} \mid q(\sigma) \neq 0\}$ et $\sigma_1 < \sigma_2 < \dots$.

Question I.2. Définissez une fonction OCaml `nonzero_occurrences : int array -> (int * int) array` qui passe de la représentation de q de la question I.1 (tableau $\llbracket q(0); q(1); \dots; q(|\mathcal{S}| - 1) \rrbracket$) à la représentation sous forme d'un tableau de paires. Cette fonction devra avoir une complexité temporelle en $O(|\mathcal{S}|)$.

Partie II. Arbres binaires

Soit $\mathcal{T}_{\mathcal{S}}$ l'ensemble des arbres binaires dont les feuilles sont en bijection avec un ensemble fini $\mathcal{S}$. Autrement dit, un arbre de $\mathcal{T}_{\mathcal{S}}$ a exactement $|\mathcal{S}|$ feuilles ; chacune de ses feuilles est étiquetée par un élément de $\mathcal{S}$; toutes les feuilles sont étiquetées par des éléments différents de $\mathcal{S}$. Ces arbres peuvent être construits comme suit :

- Une feuille étiquetée par x est notée $F(x)$. C'est un arbre de $\mathcal{T}_{\{x\}}$.
- Si g et d sont des arbres appartenant respectivement à $\mathcal{T}_{\mathcal{S}_1}$ et $\mathcal{T}_{\mathcal{S}_2}$, alors l'arbre dont le sous-arbre gauche est g et le sous-arbre droit est d , noté $N(g, d)$, est un arbre de $\mathcal{T}_{\mathcal{S}_1 \cup \mathcal{S}_2}$ à condition que $\mathcal{S}_1 \cap \mathcal{S}_2 = \emptyset$.

Remarque : un tel arbre binaire possède exactement $|\mathcal{S}| - 1$ nœuds internes. Par ailleurs, on étiquettera implicitement les arêtes par 0 ou 1 suivant qu'elles mènent à un sous-arbre gauche (0) ou droit (1).

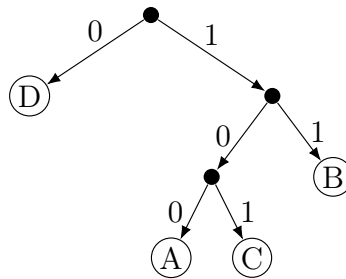


FIGURE 1 – Exemple d'arbre de $\mathcal{T}_{\{A,B,C,D\}}$.

Question II.1. Montrez que l'ensemble $\mathcal{T}_{\{A,B,C,D\}}$ contient 120 arbres.

Le type OCaml utilisé pour représenter les arbres est le suivant :

```
type tree = F of int | N of tree * tree
```

Étant donné un arbre t de $\mathcal{T}_{\mathcal{S}}$ et un élément σ de $\mathcal{S}$, on note $\ell_t(\sigma)$ la profondeur de la feuille étiquetée par σ , c'est-à-dire le nombre d'arêtes entre la racine de t et cette feuille. Par exemple, avec l'arbre de la figure 1, on a $\ell_t(A) = 3$.

Comme précédemment, soit q une fonction de $\mathcal{S}$ dans $\mathbb{N}$. Pour tout arbre $t \in \mathcal{T}_{\mathcal{S}'}$ avec $\mathcal{S}' \subseteq \mathcal{S}$, on note $c_q(t)$ la somme pondérée suivante :

$$c_q(t) = \sum_{\sigma \in \mathcal{S}'} q(\sigma) \ell_t(\sigma).$$

Question II.2. Supposons $\mathcal{S} = [0, n-1]$. Définissez une fonction OCaml `cq : tree -> int array -> int` qui reçoit deux arguments, un arbre $t \in \mathcal{T}_{\mathcal{S}}$ et un tableau $\llbracket q(0); q(1); \dots; q(n-1) \rrbracket$ représentant q , et qui renvoie la valeur de $c_q(t)$. On cherchera à écrire une fonction efficace. Donnez et justifiez sa complexité temporelle.

Étant donné un arbre t de $\mathcal{T}_{\mathcal{S}}$, on représente une lettre $\sigma \in \mathcal{S}$ par la séquence des bits obtenus en parcourant t de la racine vers la feuille $F(\sigma)$. Par exemple, dans l'arbre de $\mathcal{T}_{\{A,B,C,D\}}$ de la figure 1, D est représenté par 0 tandis que A est représenté par 100.

Question II.3. Définissez une fonction OCaml `get_path : int -> tree -> int list` qui reçoit en argument un entier $\sigma \in \mathcal{S}$ et un arbre $t \in \mathcal{T}_{\mathcal{S}}$ et qui renvoie la liste de 0 et 1 représentant σ dans t . Donnez et justifiez la complexité temporelle de cette fonction.

Considérons les séquences de bits définies de la façon suivante. Elles commencent par $k \geq 0$ bits valant 1, suivis d'un bit à 0, suivis de k bits arbitraires, ce que l'on notera $1^k 0 (0 \mid 1)^k$. Les dix premières séquences par ordre lexicographique sont donc 0, 100, 101, 11000, 11001, 11010, 11011, 1110000, 1110001, 1110010. L'arbre dont les branches constituent ces séquences et dont les feuilles sont étiquetées de gauche à droite par des entiers croissants commence comme illustré sur la figure 2.

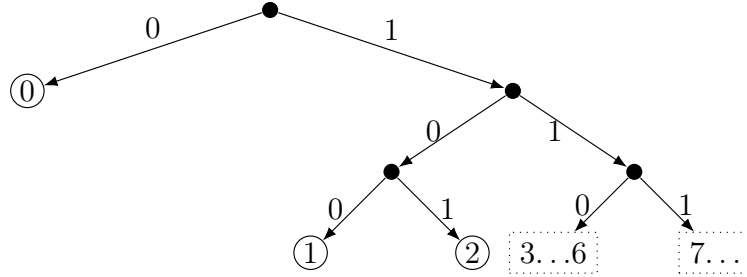


FIGURE 2 – Arbre des séquences $1^k 0 (0 \mid 1)^k$.

Remarque : l'étiquette de chaque feuille correspond à l'indice de la séquence quand elles sont triées par ordre lexicographique. Les séquences croissant indéfiniment, l'arbre est *a priori* infini. C'est pourquoi la question suivante se limite aux séquences de bits $1^k 0 (0 \mid 1)^k$ pour lesquelles k n'excède pas une certaine borne ℓ , afin d'obtenir un arbre de profondeur bornée $2\ell + 1$.

Question II.4. Définissez une fonction OCaml `integers : int -> tree` qui prend un entier $\ell \geq 0$ en argument et renvoie l'arbre dont les branches sont les séquences de la forme $1^k 0 (0 \mid 1)^k$ avec $k \leq \ell$, ainsi que la séquence $1^{\ell+1}$. Les feuilles seront étiquetées de gauche à droite par les entiers 0, 1, 2, etc.

Partie III. Codes préfixes

Étant donné un alphabet $\mathcal{S}$ et un arbre t de $\mathcal{T}_{\mathcal{S}}$, un mot de $\mathcal{S}^*$ peut être représenté par la concaténation des séquences de bits représentant chacune de ses lettres dans t , de la gauche vers la droite. Supposons que l'arbre t est l'exemple de la figure 1. Le mot « ADBDCD » est alors représenté par la séquence de bits 100|0|11|0|101|0. Remarque : les barres verticales ne servent qu'à rendre l'exemple plus lisible ; elles ne font pas partie de la séquence de bits et ne sont pas nécessaires pour retrouver le mot initial.

Question III.1. Soit t un arbre de $\mathcal{T}_{\mathcal{S}}$. Étant donnée une séquence de bits, montrez qu'il existe au plus un mot de $\mathcal{S}^*$ qui est représenté par cette séquence.

Question III.2. Supposons $\mathcal{S} \subseteq \mathbb{N}$. Définissez une fonction OCaml `decomp1 : int list -> tree -> int list` qui reçoit en argument une liste d'entiers 0 ou 1 et un arbre de $\mathcal{T}_{\mathcal{S}}$ et qui renvoie la liste des éléments de $\mathcal{S}$ représentée par cette liste de bits. On sera attentif au cas où la liste en argument ne représente aucun mot de $\mathcal{S}^*$. Donnez et justifiez la complexité temporelle de cette fonction.

Cette fonction est dite de *décompression*. Supposons que $\mathcal{S}$ est $\{A, B, C, D\}$ et que la fonction q donne le nombre d'occurrences de chaque lettre dans le mot « ADBDCD », par exemple $q(D) = 3$. Dans ce cas, $c_q(t)$ vaut 11 pour l'arbre exemple de la Figure 1. Il s'agit de la longueur de la séquence 10001101010 représentant le mot « ADBDCD ». Plus généralement, $c_q(t)$ est la longueur de n'importe quelle séquence représentant un mot de $\mathcal{S}^q$ compressé avec l'arbre t . Il s'avère qu'il n'existe aucun arbre $t' \in \mathcal{T}_{\mathcal{S}}$ tel que $c_q(t') < 11$; t est donc optimal.

Étant donnée une fonction q , l'objectif de la partie suivante sera de construire un des arbres de $\mathcal{T}_{\mathcal{S}}$ offrant la meilleure *compression* des mots de $\mathcal{S}^q$, c'est-à-dire un arbre qui minimise c_q .

A2023 – INFO MPI I

**ÉCOLE DES PONTS PARISTECH,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

Concours Mines-Télécom

CONCOURS 2023

PREMIÈRE ÉPREUVE D'INFORMATIQUE

Durée de l'épreuve : 3 heures

L'usage de la calculatrice et de tout dispositif électronique est interdit.

*Les candidats sont priés de mentionner de façon apparente
sur la première page de la copie :*

INFORMATIQUE I - MPI

L'énoncé de cette épreuve comporte 9 pages de texte.

Cette épreuve concerne uniquement les candidats de la filière MPI.

*Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé,
il le signale sur sa copie et poursuit sa composition en expliquant les raisons
des initiatives qu'il est amené à prendre.*

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence
Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.
Tout autre usage est soumis à une autorisation préalable du Concours commun Mines Ponts.



Préliminaires

Présentation du sujet

L'épreuve est composée d'un problème unique comportant 32 questions divisées en trois sections. L'objectif du problème est de construire des *listes à accès direct* : une liste à accès direct est un type de donnée abstrait qui permet, d'une part, d'empiler et de dépiler efficacement un élément en tête de liste et, d'autre part, d'accéder efficacement au k^{e} élément de la liste, pour n'importe quel indice k valide.

Dans la première section (page 1), nous étudions un système de numération : la représentation binaire gauche des entiers naturels. Dans la deuxième section (page 4), nous étudions les arbres binaires parfaits. Dans la troisième section (page 6), nous implémentons le type liste à accès direct par la structure de données concrète *liste gauche*, que nous construisons en exploitant les résultats obtenus dans les sections précédentes.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractère différentes désignera la même entité, mais du point de vue mathématique avec la police en italique (par exemple n) et du point de vue informatique avec celle en romain avec espacement fixe (par exemple `n`).

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question antérieure, même sans avoir réussi à établir ce résultat. Des rappels de programmation sont faits en annexe et peuvent être utilisés directement.

Selon les questions, il faudra coder des fonctions à l'aide du langage de programmation C exclusivement, en reprenant le prototype de fonction fourni par le sujet, ou en pseudo-code (c-à-d. dans une syntaxe souple mais conforme aux possibilités offertes par le langage C). Il est inutile de rappeler que les entêtes `<assert.h>`, `<stdbool.h>`, etc. doivent être inclus.

Quand l'énoncé demande de coder une fonction, sauf indication explicite de l'énoncé, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites. On suppose que le type `int` n'est jamais sujet à des débordements.

Le barème tient compte de la clarté des programmes : nous recommandons de choisir des noms de variables intelligibles ou encore de structurer de longs codes par des blocs ou par des fonctions auxiliaires dont on décrit le rôle. Lorsqu'une réponse en pseudo-code est permise, seule la logique de programmation est évaluée, même dans le cas où un code en C a été fourni en guise de réponse.

1. Représentation binaire gauche des entiers naturels

1.1. Mise en place

Soient m un entier naturel et N un entier naturel non nul. Il est classique d'appeler *représentation binaire standard de l'entier m sur N chiffres*, ou plus simplement *représentation standard*, toute suite finie $b = (b_n)_{0 \leq n < N}$ de longueur N telle que, pour tout indice n compris entre 0 et $N - 1$, le chiffre b_n appartient à $\{0, 1\}$ et l'égalité suivante est vérifiée

$$m = \sum_{n=0}^{N-1} b_n 2^n.$$

Première épreuve d'informatique MPI 2023

Définition : Nous appelons *représentation binaire gauche de l'entier m sur N chiffres* toute suite finie $g = (g_n)_{0 \leq n < N}$ de longueur N telle que,

- (i) pour tout indice n compris entre 0 et $N - 1$, le chiffre g_n appartient à $\{0, 1, 2\}$,
- (ii) l'égalité suivante est satisfaite

$$m = \sum_{n=0}^{N-1} g_n(2^{n+1} - 1),$$

- (iii) le chiffre « 2 » n'apparaît qu'au plus une fois parmi les chiffres $(g_n)_{0 \leq n < N}$,
- (iv) s'il existe une position p tel que le chiffre g_p est 2, alors, pour tout indice n compris entre 0 et $p - 1$, le chiffre g_n est nul.

De manière plus courte, nous parlons simplement de *représentation gauche*.

La figure 1 ci-dessous donne une représentation standard et une représentation gauche sur quatre chiffres des seize premiers entiers. Conformément à l'usage habituel, nous écrivons toute représentation, qu'elle soit standard ou gauche, sous la forme d'un mot $b_{N-1} \cdots b_0$ ou $g_{N-1} \cdots g_0$ dans lequel les chiffres de poids faibles sont écrits à droite.

Entier	Repr. standard	Repr. gauche	Entier	Repr. standard	Repr. gauche
0	0000	0000	8	1000	0101
1	0001	0001	9	1001	0102
2	0010	0002	10	1010	0110
3	0011	0010	11	1011	0111
4	0100	0011	12	1100	0112
5	0101	0012	13	1101	0120
6	0110	0020	14	1110	0200
7	0111	0100	15	1111	1000

FIGURE 1 – Représentations standard et gauche des seize premiers entiers

□ 1 – Soit c un entier. Donner la représentation standard de l'entier dont une représentation gauche est $10 \cdots 0$ (avec c chiffres nuls) en justifiant sommairement. Faire de même avec l'entier dont une représentation gauche est $20 \cdots 0$ (avec c chiffres nuls).

□ 2 – Déterminer, en justifiant, le plus grand entier naturel M_N qui admet une représentation gauche sur N chiffres. Préciser la représentation gauche de cet entier.

Définition : Soit n_0 un indice compris entre 0 et $N - 1$. On dit que l'indice n_0 est la *position du chiffre de plus fort poids* d'une représentation $g_{N-1} \cdots g_0$ si l'indice n_0 est le plus petit entier tel que, pour tout indice $n > n_0$, le chiffre g_n est nul. On appelle le chiffre g_{n_0} le *chiffre de plus fort poids*.

□ 3 – Soient N un entier naturel non nul, $g = g_{N-1} \cdots g_0$ et $h = h_{N-1} \cdots h_0$ deux représentations gauches d'un même entier m . Démontrer que les chiffres de plus fort poids de g et de h sont de même valeur et à la même position.

□ 4 – Démontrer que tout entier appartenant à l'intervalle $\llbracket 0, M_N \rrbracket$, où l'entier M_N a été introduit à la question 2, ne possède au plus qu'une seule représentation gauche sur N chiffres.

Première épreuve d'informatique MPI 2023

Indication C : L'entier N est déclaré comme constante globale. Nous utilisons la structure C déclarée comme suit pour écrire la représentation gauche sur N chiffres d'un entier $g_{N-1} \cdots g_0$:

```

1.  const int N = 8;
2.
3.  struct RepGauche {
4.      int position;
5.      bool chiffres[N];
6.  };
7.  typedef struct RepGauche rg;

```

Le champ `position` repère la position éventuelle du chiffre 2; il vaut -1 au cas où le chiffre 2 n'apparaît pas. Pour tout indice n compris entre 0 et $N - 1$, la n^{e} case du champ `chiffres` vaut `true` si le chiffre g_n est non-nul et vaut `false` sinon.

Par exemple, les entiers 15 et 21 ont pour représentation gauche les variables `entier_15` et `entier_21` suivantes :

```

8.  rg entier_15 = { .position = -1,
9.                  .chiffres = { 0, 0, 0, 1, 0, 0, 0, 0 } };
10. rg entier_21 = { .position = 1,
11.                 .chiffres = { 0, 1, 0, 1, 0, 0, 0, 0 } };

```

□ 5 – Formaliser sous la forme d'un ou de plusieurs invariants le fait qu'une valeur C de type **rg** est la représentation gauche d'un entier.

□ 6 – Écrire une fonction C **int** `rg_to_int(rg g)`, qui renvoie l'entier dont g est la représentation gauche. On supposera que l'invariant de la question 5 est satisfait.

1.2. Incrémentation et décrémentation

Nous proposons l'algorithme suivant :

ALGORITHME MYSTÈRE :

Entrée : Représentation gauche $g = g_{N-1} \cdots g_0$ d'un certain entier m .

Effet :

- Si aucun des chiffres $(g_n)_{0 \leq n < N}$ ne vaut 2, changer le chiffre g_0 en $g_0 + 1$.
- Sinon, en notant p la position du chiffre 2, changer le chiffre g_p en 0 et le chiffre g_{p+1} en $g_{p+1} + 1$.

Nous notons m' l'entier dont la représentation gauche est g après exécution de l'algorithme.

FIGURE 2 – Un algorithme

□ 7 – Vérifier que l'invariant de la question 5 n'est pas rompu par l'algorithme mystère (cf. figure 2). Avec les notations m et m' de la figure 2, caractériser, en fonction de l'entier m , la valeur de l'entier m' .

Première épreuve d'informatique MPI 2023

- 8 – Écrire une fonction C `bool rg_incr(rg *s)` dont la spécification suit :
- Précondition* : La variable s est un pointeur vers la représentation gauche d'un entier m .
- Effet* : La valeur pointée par s est modifiée afin de représenter l'entier $m + 1$.
- Valeur de retour* : Booléen `true` si l'incrément de m peut avoir lieu et `false` si un débordement se produit car $m + 1$ n'est pas représentable sur le même nombre de chiffres.
- 9 – Calculer la complexité en temps dans le pire des cas de la fonction `rg_incr` en fonction de N . Comparer avec la complexité de la même opération sur la représentation standard.
- 10 – Écrire une fonction C `bool rg_decr(rg *s)` dont la spécification suit :
- Précondition* : Le pointeur s désigne la représentation gauche d'un entier m .
- Effet* : La valeur pointée par s est modifiée afin de représenter l'entier $m - 1$.
- Valeur de retour* : Booléen `true` si la décrémentation de m peut avoir lieu et `false` si un débordement se produit car m est nul.
- Il est recommandé d'expliquer son intention avant de donner son code.

2. Arbres binaires parfaits

2.1. Opérations sur les arbres binaires

Indication C : Nous représentons des arbres binaires à valeurs entières au moyen du type C `arb` suivant, qui est un pointeur vers une structure contenant la valeur entière dans le champ `valeur` et les deux fils dans les champs `fils_g` et `fils_d`. L'arbre vide se représente par le pointeur `NULL`.

```

12.  typedef struct Noeud *arb;
13.  struct Noeud {
14.      int valeur;
15.      arb fils_g;
16.      arb fils_d;
17.  };

```

L'arbre vide est, par convention, de hauteur -1 .

- 11 – Écrire une fonction C `int hauteur(arb a)` qui calcule la hauteur de l'arbre a .
- 12 – Écrire une fonction C `arb noeud(int v, arb ag, arb ad)` qui construit un arbre dont la racine a pour étiquette v , dont le fils gauche est a_g et le fils droit est a_d . Dans cette question, il est demandé de se défendre, par le truchement d'une assertion, de toute erreur liée à un échec d'allocation dynamique de mémoire.

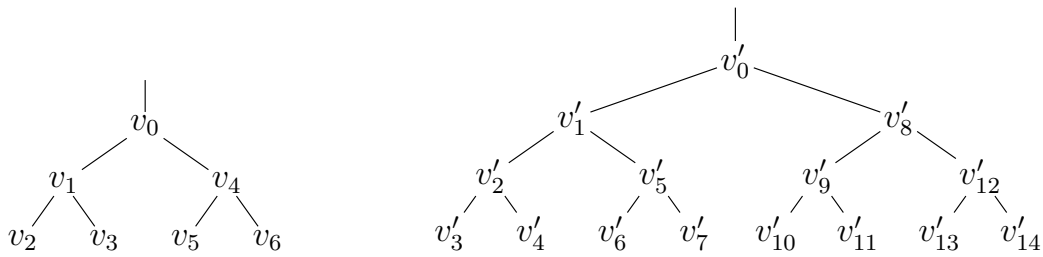


FIGURE 3 – Arbres binaires parfaits de hauteur 2 et de hauteur 3.

2.2. Arbres parfaits

Définition : Un *arbre binaire parfait*, ou simplement *arbre parfait*, est un arbre binaire dont tous les nœuds internes ont exactement deux fils, dont toutes les feuilles sont à la même profondeur et dont les nœuds sont étiquetés par des valeurs entières.

Des exemples d'arbres binaires parfaits sont donnés figure 3.

- 13 – Démontrer que tout arbre binaire parfait de hauteur n possède $2^{n+1} - 1$ nœuds.
- 14 – Écrire une fonction C `bool est_parfait(arb a, int n)` dont la spécification suit :
Précondition : Le pointeur a désigne la racine d'un arbre binaire (dont les nœuds sont bien tous distincts).
Valeur de retour : Booléen `true` si l'arbre pointé est parfait de hauteur n et `false` sinon.
- 15 – Calculer la complexité en temps dans le pire des cas de l'exécution de `est_parfait(a, n)` en fonction de l'entier n .

2.3. Opérations sur les arbres parfaits

Définition : Soient S une structure de données qui permet de stocker une collection d'entiers et t le cardinal de S . Nous disons que la structure de données S est à *accès direct* s'il existe une manière systématique de numéroter chaque élément de S entre 0 et $t - 1$ et s'il existe deux primitives, `acces` et `modif`, qui permettent respectivement de consulter et de modifier n'importe quel élément de S en temps logarithmique en t à partir seulement de son numéro.

Nous souhaitons vérifier qu'un arbre parfait est une structure de données à accès direct. Nous numérotions les éléments d'un arbre parfait en utilisant l'ordre préfixe de gauche à droite de l'arbre (comme illustré dans la figure 3).

- 16 – Écrire une fonction C `arb arb_trouve(arb a, int n, int k)` ainsi spécifiée :
Précondition : Le pointeur a désigne la racine d'un arbre binaire parfait de hauteur n . L'entier k satisfait les inégalités $0 \leq k < 2^{n+1} - 1$.
Valeur de retour : Pointeur vers le k^e nœud de l'arbre dans l'ordre préfixe.

Il est rappelé que la racine d'un arbre est à profondeur 0. Nous donnons la formule de sommation, valable pour tout réel $a \neq 1$ et tout entier t ,

$$\sum_{k=0}^t k \cdot a^k = \frac{((a-1)t-1)a^{t+1} + a}{(a-1)^2}.$$

Première épreuve d'informatique MPI 2023

- 17 – Nous appelons P la profondeur d'un nœud choisi aléatoirement et uniformément parmi les $2^{n+1} - 1$ nœuds d'un arbre binaire parfait de hauteur n . Calculer l'espérance $\mathbb{E}(P)$ de la variable aléatoire P .
- 18 – Déterminer la complexité moyenne en temps de l'instruction `arb_trouve(a, n, k)` lorsque la hauteur n et l'arbre a sont fixés et le numéro k est choisi aléatoirement et uniformément dans l'intervalle $\llbracket 0, 2^{n+1} - 2 \rrbracket$.
- 19 – Écrire une fonction C `int arb_acces(arb a, int n, int k)` qui renvoie la k^{e} valeur de l'arbre a de hauteur n ainsi qu'une fonction C `void arb_modif(arb a, int n, int k, int v)` qui remplace la k^{e} valeur de l'arbre a de hauteur n par la valeur v . Dire finalement si la structure de données arbre binaire parfait est à accès direct.

3. Listes gauches

Les arbres binaires parfaits seuls ne permettent de représenter que des collections d'entiers dont le cardinal est de la forme $2^{n+1} - 1$ où n est entier naturel. Afin de représenter des collections de cardinal quelconque, nous introduisons des suites d'arbres parfaits, aux hauteurs strictement croissantes et savamment choisies.

Définition : Nous appelons *liste binaire gauche de cardinal m sur N arbres* la structure de données constituée de $N + 1$ arbres binaires parfaits $((a_n)_{0 \leq n < N}, e)$ comme suit. Nous décomposons l'entier m selon sa représentation binaire gauche sur N chiffres : $g_{N-1} \dots g_0$. Pour tout indice n compris entre 0 et $N - 1$, si le chiffre g_n n'est pas nul, l'arbre binaire parfait a_n est un arbre de hauteur n , sinon il s'agit de l'arbre vide. Si le chiffre 2 apparaît parmi les chiffres de la représentation gauche de m et si l'indice p est sa position, alors l'arbre e est un arbre binaire parfait de hauteur p , sinon l'arbre e est l'arbre vide. De manière plus courte, nous parlons simplement de *liste gauche*.

Une liste binaire gauche de cardinal m sur N arbres permet de stocker une collection de m éléments entiers. Il apparaît qu'avec N arbres, une liste binaire gauche peut stocker jusqu'à M_N entiers (où l'entier M_N a été introduit à la question 2) et que m est inférieur à M_N .

Indication C : Nous adoptons le type C suivant

```
18. struct ListeGauche {
19.     int hauteur_e;
20.     arb extra;
21.     int nb_arbres;
22.     arb *arbres;
23. };
24. typedef struct ListeGauche lg;
```

Le champ `hauteur_e` contient la hauteur p de l'arbre exceptionnel e . Le champ `extra` désigne la racine de l'arbre exceptionnel e . Le champ `nb_arbres` contient l'entier N . Enfin, le champ `arbres` désigne un tableau de N pointeurs vers les arbres $(a_n)_{0 \leq n < N}$ qui a été alloué dynamiquement.

3.1. Opérations simples sur les listes gauches

□ 20 – Écrire une fonction C `lg lg_init(int N)` qui renvoie une liste gauche de cardinal nul sur N arbres.

□ 21 – Écrire une fonction C `int lg_card(lg l)` qui calcule le cardinal m de la liste gauche ℓ .

Définition : Afin de numéroter l'ensemble des éléments d'une liste gauche $\ell = ((a_n)_{0 \leq n < N}, e)$, nous parcourons d'abord les éléments de l'arbre exceptionnel e dans l'ordre préfixe, puis nous parcourons les éléments des arbres $a_0, a_1, \dots, a_{N-1}$ dans l'ordre préfixe. Les numéros sont attribués aux valeurs rencontrées par ordre de première rencontre.

□ 22 – Écrire une fonction C `arb lg_trouve(lg l, int k)` qui renvoie le k^{e} nœud de la liste gauche ℓ . On supposera que k est un indice valide ($0 \leq k < m$).

□ 23 – Calculer la complexité en temps dans le pire des cas de `lg_trouve l` en fonction de la capacité maximale M_N de la liste gauche ℓ .

□ 24 – Écrire une fonction C `int lg_acces(lg l, int k)` qui renvoie la k^{e} valeur de la liste gauche ℓ ainsi qu'une fonction C `void lg_modif(lg l, int k, int v)` qui remplace la k^{e} valeur de la liste gauche ℓ par la valeur v .

3.2. Ajout et suppression en tête de liste gauche

□ 25 – Soient v une valeur entière et $\ell = ((a_n)_{0 \leq n < N}, e)$ une liste gauche de cardinal m , avec $m < M_N$, qui contient les éléments $v_1, \dots, v_m$ dans cet ordre. Décrire, en fonction de la liste gauche ℓ , la liste gauche $\ell' = ((a'_n)_{0 \leq n < N}, e')$ de cardinal $m + 1$ dont les éléments sont $v, v_1, \dots, v_m$ dans cet ordre. En déduire le principe d'une fonction C `bool lg_empile(int v, lg l)` réalisant l'insertion de la valeur v en tête de la liste gauche ℓ où le booléen résultat vaut `true` si l'ajout a eu lieu et vaut `false` si un débordement de capacité de la liste se produit. On ne demande pas le code complet de cette fonction.

□ 26 – Déterminer la complexité en temps dans le pire des cas de la fonction `lg_empile`.

□ 27 – Donner le principe d'une fonction C `bool lg_depille(int *w, lg l)` réalisant le retrait de l'élément de tête de la liste gauche ℓ et son affectation à l'adresse w . Le booléen résultat vaut `true` si le retrait a eu lieu et vaut `false` si l'opération a échoué en raison d'une liste vide. On ne demande pas le code complet.

□ 28 – Donner la complexité en temps dans le pire des cas de la fonction `lg_depille`.

□ 29 – Discuter la possibilité d'obtenir une complexité plus faible à la question 28, quitte à modifier légèrement la définition du type `lg`.

□ 30 – Soit N un entier et $\ell = ((a_n)_{0 \leq n < N}, e)$ une liste gauche. Discuter la possibilité de modifier en place et avec une faible complexité en temps la liste gauche ℓ de sorte que le nombre d'arbres N devienne $N + 1$ et que les mêmes éléments demeurent dans la liste.

3.3. Utilisation concurrente des listes gauches

Dans cette sous-section, on raisonne sur les algorithmes en supposant que les fils d'exécution peuvent s'entrelacer mais que les instructions d'un même fil s'exécutent dans l'ordre du programme. L'entête `#include <pthread.h>` a été déclarée ; la syntaxe de certaines fonctions s'y rattachant est rappelée en annexe.

□ 31 – Deux fils d'exécution distincts exécutent la fonction `lg_empile` sur la même liste gauche. Montrer qu'une course critique advient de leur exécution concurrente et que la cohérence de ladite liste gauche n'est pas garantie, autrement dit que certains invariants qui caractérisent la bonne formation d'une liste gauche peuvent être violés à l'issue des exécutions.

Nous nous intéressons finalement au *problème des producteurs et des consommateurs* qui s'échangent des entiers au travers d'un tampon, ici constitué par une unique liste gauche ℓ à N arbres, l'entier N étant fixé à l'avance. Les producteurs écrivent dans la liste gauche ℓ en empilant un entier à la fois en tête, à condition que la liste gauche ℓ ne soit pas pleine ; les consommateurs vident la liste gauche ℓ en dépilant l'entier en tête de la liste, à condition que la liste gauche ℓ ne soit pas vide.

Un seul agent peut accéder au tampon à la fois. Lorsqu'un consommateur souhaite supprimer une donnée alors que le tampon est vide, il est mis en attente ; lorsqu'un producteur souhaite écrire une donnée alors que le tampon est plein, il est mis en attente.

□ 32 – Décrire une solution au problème des producteurs et des consommateurs en s'appuyant sur un verrou et sur deux sémaphores. Détailler sous la forme de code C ou bien de pseudo-code ce que font les producteurs et les consommateurs.

* *

*

Les listes binaires gauches, ou *skew binary lists*, ont été inventées par Eugene Myers en 1983.

A. Rappels de programmation en C

L'expression $1 \ll n$ représente le décalage de la valeur 1 sur n bits : elle a pour valeur l'entier 2^n .

Le type **pthread_t** désigne des fils d'exécution.

L'instruction `pthread_create(pthread_t *th_id, NULL, &ma_fonction, void *args)` crée un nouveau fil d'exécution qui appelle la fonction `ma_fonction` sur le ou les arguments désignés par `args` et qui s'exécute simultanément avec le fil d'exécution appelant.

L'instruction `pthread_join(th_id, NULL)` suspend l'exécution du fil d'exécution appelant jusqu'à ce que le fil d'exécution identifié par `th_id` achève son exécution.

Le type **pthread_mutex_t** désigne des verrous.

L'instruction `pthread_mutex_lock(&v)` verrouille le verrou `v`.

L'instruction `pthread_mutex_unlock(&v)` déverrouille le verrou `v`.

Le type **sem_t** désigne des sémaphores.

L'instruction `sem_init(&s, 0, v)` initialise le sémaphore `s` à la valeur v (avec $v \geq 0$).

L'instruction `sem_wait(&s)` décrémente le compteur du sémaphore `s` : si le compteur est toujours positif, l'appel se termine ; sinon le fil d'exécution appelant est bloqué.

L'instruction `sem_post(&s)` incrémente le compteur du sémaphore `s` et, si le compteur redevient strictement positif, réveille un fil d'exécution bloqué sur `s`.

FIN DE L'ÉPREUVE

SESSION 2023



MPI5IN

ÉPREUVE MUTUALISÉE AVEC E3A-POLYTECH**ÉPREUVE SPÉCIFIQUE - FILIÈRE MPI**

INFORMATIQUE**Durée : 4 heures**

N.B. : le candidat attachera la plus grande importance à la clarté, à la précision et à la concision de la rédaction. Si un candidat est amené à repérer ce qui peut lui sembler être une erreur d'énoncé, il le signalera sur sa copie et devra poursuivre sa composition en expliquant les raisons des initiatives qu'il a été amené à prendre.

RAPPEL DES CONSIGNES

- Utiliser uniquement un stylo noir ou bleu foncé non effaçable pour la rédaction de votre composition ; d'autres couleurs, excepté le vert, peuvent être utilisées, mais exclusivement pour les schémas et la mise en évidence des résultats.
 - Ne pas utiliser de correcteur.
 - Écrire le mot *FIN* à la fin de votre composition.
-

Les calculatrices sont interdites.

Le sujet est composé de trois parties, toutes indépendantes.

Partie I - Palindromes

En langue française, "ressasser" est le mot palindrome le plus long, tandis qu'il semble que "saippua-kauppias" soit le plus long mot palindrome au monde, désignant un marchand de savon en Finlande. L'objet de cette partie est de compter le nombre de palindromes présents dans un mot donné.

Soit Σ un alphabet fini contenant au moins deux lettres. On note $u = u_0 \cdots u_{n-1}$ un *mot* sur Σ , composé de n lettres $u_i \in \Sigma, i \in \llbracket 0, n-1 \rrbracket$. La longueur de u est notée $|u|$. Pour $0 \leq i < j \leq n$, on note $u[i, j]$ le mot $u_i \cdots u_{j-1}$. L'ensemble des mots construits sur Σ et contenant le mot vide ϵ est noté Σ^* .

Définition 1 (Miroir)

Soit $u \in \Sigma^*$. Le *miroir* de $u = u_0 \cdots u_{n-1}$, noté $\bar{u}$, est le mot $\bar{u} = u_{n-1} \cdots u_0$. Par convention $\bar{\epsilon} = \epsilon$.

Définition 2 (Palindrome)

$u \in \Sigma^*$ est un *palindrome* si et seulement si $u = \bar{u}$.

Par convention, le mot vide ϵ n'est pas considéré comme un palindrome.

On dira qu'un palindrome u est *pair* (respectivement *impair*) lorsque sa longueur $|u|$ est paire (resp. impaire).

On recherche donc dans cette partie le nombre de palindromes facteurs d'un mot $u \in \Sigma^*$ (comptés avec les multiplicités éventuelles), soit le cardinal de l'ensemble $\{(i, j), 0 \leq i < j \leq |u|, u[i, j] = \overline{u[i, j]}\}$. Dit autrement, on recherche le nombre de palindromes contenus dans u .

Q1. Si $\Sigma = \{a, b\}$, donner le nombre de palindromes contenus dans le mot $u = babb$.

En parcourant naïvement les lettres d'un mot u donné, on peut proposer un algorithme en pseudo-code permettant de compter le nombre palindromes de u (**algorithme 1**).

Algorithme 1 - Décompte naïf du nombre de palindromes contenus dans un mot donné.

Entrées : Un mot u .

Sorties : Le nombre de palindromes contenus dans u .

début

```

    nb = 0
    pour i=0 à |u|-1 faire
        pour j=0 à |u|-1 faire
            estPalindrome=True
            pour k=i à j-1 faire
                si u[i] ≠ u[j-k-1] alors
                    estPalindrome=False
                Sortir du pour k
            si estPalindrome alors
                nb = nb+1
    retourner nb
```

Q2. Évaluer la complexité au pire des cas de l'**algorithme 1** en fonction de $|u|$.

On souhaite bien sûr améliorer cette première idée. Pour ce faire, on utilise tout d'abord le paradigme de la programmation dynamique.

Pour $u \in \Sigma^*$, on définit un tableau de booléens P de taille $(|u|+1) \times (|u|+1)$, $P[i][j]$ étant vrai si $u[i, j]$ est un palindrome. On a donc pour tout $i \in \llbracket 0, |u|-1 \rrbracket$, $P[i][i+1] = \text{True}$.

Q3. Soit $u[i, j]$ un mot. À quelles conditions sur u_i , u_{j-1} et $u[i+1, j-1]$ le mot $u[i, j]$ est-il un palindrome ?

Q4. En déduire une relation de récurrence vérifiée par les coefficients de P .

Q5. Écrire un algorithme de programmation dynamique en pseudo-code résolvant le problème. Évaluer sa complexité.

On insère maintenant entre chaque paire de lettres de u , ainsi qu'au début et à la fin du mot, un symbole spécial noté $\#$. On appelle ce nouveau mot $u^\#$. Ainsi, le mot $u = abba$ se transforme en $u^\# = \#a\#b\#b\#a\#$.

Q6. Montrer que les palindromes de $u^\#$, s'ils existent, sont tous impairs.

Q7. Soit v un palindrome de longueur $2k + 1$ de u , $k \in \mathbb{N}$. On construit le mot $u^\#$. Donner les deux palindromes correspondants à v dans $u^\#$.

Q8. Même question si v est un palindrome de longueur $2k$ de u .

Q9. En déduire une stratégie de recherche de tous les palindromes de u .

On voit que les palindromes impairs sont importants. On va donc construire un algorithme de recherche de ce type de palindrome.

Définition 3 (Rayon d'un palindrome)

Soient $u \in \Sigma^*$ et $i \in \llbracket 0, |u| - 1 \rrbracket$. On dit qu'il existe un *palindrome centré en i de rayon $\rho > 0$* si :

- (i). $i - \rho \geq 0$,
- (ii). $i + \rho + 1 \leq |u|$,
- (iii). $u[i - \rho, i + \rho + 1]$ est un palindrome.

Le *rayon maximal* $\hat{\rho}_i$ d'un palindrome centré en i est le plus grand rayon d'un palindrome centré en i .

Par exemple, si $u = abbababab$ et $i = 4$, alors b est un palindrome centré en 4 de rayon 0, aba est un palindrome centré en 4 de rayon 1, $babab$ est un palindrome centré en 4 de rayon 2 et c'est le plus grand, donc $\hat{\rho}_4 = 2$.

Q10. En remarquant qu'un palindrome impair est centré sur une lettre (ou un symbole spécial dans le cas de $u^\#$), proposer un algorithme en pseudo-code permettant de compter le nombre de palindromes impairs d'un mot u . Évaluer sa complexité.

Q11. Soient $u \in \Sigma^*$, $i \in \llbracket 0, |u| - 1 \rrbracket$ et $0 < j \in \llbracket 1, \hat{\rho}_i \rrbracket$. En exploitant les différentes symétries, montrer qu'il existe un palindrome centré en $i + j$ de rayon $\min(\hat{\rho}_i - j, \hat{\rho}_{i-j})$. En déduire $\hat{\rho}_{i+j} \geq \min(\hat{\rho}_i - j, \hat{\rho}_{i-j})$. Préciser à quelle condition il y a égalité.

En utilisant cette remarque, on développe un algorithme, dit de Manacher, qui construit un tableau d'entiers T permettant de compter le nombre de palindromes d'un mot u . Plus précisément, pour chaque position $i \in \llbracket 1, |u| - 1 \rrbracket$, $T[i]$ indique le rayon maximal $\hat{\rho}_i$, donc tel que la sous-chaîne de $i - \rho_i$ à $i - \rho_i + 1$ est un palindrome. L'algorithme 2 consiste à incrémenter $T[i]$ jusqu'à trouver le plus grand palindrome $u[i - T[i], i + T[i] + 1]$ centré en i .

Algorithme 2 - Algorithme de Manacher

Entrées : Un mot u

Sorties : Un tableau T .

début

Initialiser un tableau T de $|u|$ cases, initialisées à 0

$k=0$

pour $i = 1$ à $|u| - 1$ **faire**

$j = i - k$

si $T[k] \geq j$ **alors**

$T[i] = \min(T[k - j], T[k] - j)$

tant que $(i - (T[i] + 1)) \geq 0$ **ET** $(i + T[i] + 1) < |u|$ **ET** $(u[i - (T[i] + 1)] = u[i + T[i] + 1])$ **faire**

$T[i] = T[i] + 1$

$k = i$

retourner T

Q12. Comment trouver à partir de cet algorithme le nombre de palindromes de u ?

Q13. Quelle est la complexité de cet algorithme ? Justifier.

Partie II - Traversée de rivière

Cette partie comporte des questions nécessitant un **code OCaml**.

Dans une vallée des Alpes, un passage à gué fait de cailloux permet de traverser la rivière. Deux groupes de randonneurs arrivent simultanément sur les berges gauche et droite de cette rivière et veulent la traverser. Le chemin étant très étroit, une seule personne peut se trouver sur chaque caillou de ce chemin (**figure 1**). Un randonneur sur la berge de gauche peut avancer d'un caillou (vers la droite sur la **figure 1**) et sauter par dessus le randonneur devant lui (un caillou à droite) si le caillou où il atterit est libre. De même, chaque randonneur de la berge de droite peut avancer d'un caillou (vers la gauche sur la **figure 1**) et sauter par dessus le randonneur devant lui, dans la mesure où le caillou sur lequel il atterit est libre. Une fois engagés, les randonneurs ne peuvent pas faire marche arrière. De plus, pour simplifier, on suppose qu'une fois tous les randonneurs sur le chemin, il ne reste qu'un caillou de libre.



Figure 1 - Les randonneurs et le chemin de cailloux

Le chemin de cailloux est défini par un tableau d'entiers :

```
type chemin_caillou = int array
```

Dans ce tableau, un randonneur venant de la berge de gauche est représenté par un 1, un randonneur issu de la berge de droite par un 2 et un caillou libre par un 0.

- Q14.** Écrire une fonction de signature `caillou_vide : chemin_caillou -> int` qui détermine la position du caillou inoccupé.
- Q15.** Écrire une fonction de signature `echange : chemin_caillou -> int -> int -> chemin_caillou` qui permute les valeurs codées sur deux cailloux. Le tableau d'entiers initial représentant le chemin n'est pas modifié. On pourra utiliser ici la fonction `copy` du module `Array`.
- Q16.** Écrire une fonction de signature `randonneurG_avance : chemin_caillou -> bool` qui teste si, parmi les randonneurs venant de la berge de gauche, il en existe un qui puisse avancer (vers la droite).
- Q17.** Écrire une fonction de signature `randonneurG_saute : chemin_caillou -> bool` qui teste si, parmi les randonneurs venant de la berge de gauche, il en existe un qui puisse sauter (vers la droite) au-dessus d'un randonneur.

On supposera dans la suite les fonctions de signature `randonneurD_avance : chemin_caillou -> bool` et `randonneurD_saute : chemin_caillou -> bool` écrites de manière similaire pour les randonneurs venant de la berge de droite.